

Национальный исследовательский университет ИТМО
Факультет компьютерных технологий и управления
Факультет программной инженерии и компьютерной техники

Отчет по практическому заданию №2
по курсу «Языки программирования»
Вариант: 4

Выполнил студент группы Р4119 _____ Н.Ю. Ковалев
(подпись)

Руководитель _____ Ю. Д. Кореньков
(подпись)

Санкт-Петербург
2025

Цели

- Реализовать модуль формирования графов потока управления (CFG) на основе уже полученных деревьев разбора и подготовить визуальные документы, которые подтверждают правильную структуру управления в подпрограммах.
- Продемонстрировать работу модуля тестовой программой, которая принимает набор входных файлов, используя результаты разбора из лабораторной 1, и сохраняет CFG, граф операций и граф вызовов в выходном каталоге.

Задачи

1. Описать набор структур: описание подпрограмм (ProgramFunction), узлов CFG (CFGNode) и операций (FlowOperation) вместе с векторам данных, отражающих деревья операций и связи между узлами.
2. Расширить модуль анализа, который итерирует по деревьям разбора, вызывает build_cfg_for_function и формирует ProgramFunction.meta.cfg, связывая каждый базовый блок с операциями, деревом операций и предикатами.
3. Собрать CLI lab2_cfg (Lab2/main.c), который принимает файлы, --outdir, --emit-cfg и другие опции, передаёт поток TSParser c tree_sitter_v2lang_test() и формирует метаинформацию о графах.
4. Написать тестовый сценарий (Lab2/generate_cfgs.sh), запускающий компиляцию и генерацию .dot/.pdf, включая агрегацию all_functions.dot и call-graph *.callgraph.*.
5. Зафиксировать обнаруженные ошибки (парсинга и логики) в stderr и в результатах анализа, а также сформировать представление графов вызовов по всем файлам.

Описание работы

lab2_cfg читает исходные файлы, парсит их через Tree-sitter из первой лабораторной (Lab1/src/parser.c), собирает funcDef и заполняет ProgramFunction (имя, сигнатура, источник). Затем он вызывает build_cfg_for_function (из flow.c), который идёт по дереву разбора и создаёт CFG, CFGNode с операциями, связями (cfg_add_edge, cfg_node_add_operation) и снабжает каждый узел деревом операций (FlowOperation). При --emit-cfg пишутся DOT-графы для каждой подпрограммы и агрегированный call-graph (callgraph.dot/.csv), --outdir задаёт каталог вывода, а --emit-cfg разрешает генерацию файлов. Ошибки (неустраняемые синтаксические/файловые) выводятся тестовой программой в stderr.

Аспекты реализации

- **Основные структуры.** В Lab2/flow.h оформлены CFG, CFGNode, FlowOperation и вспомогательные ExprInfo/BinaryKind/CondKind, которые объединяют информацию об узлах, операциях и условных переходах.
- **Обработка операций.** В flow.c функции flow_operation_new, cfg_node_add_operation, deduce_binary_kind и parse_operands парсят строки из AST-комментариев (Assign(...), Call(...), Expr), заполняют FlowOperation (левый/правый операнд, аргументы вызовов, тип ветвления) и прикрепляют операции к узлам CFG. Пример с deduce_binary_kind:

```
static BinaryKind deduce_binary_kind(const char *line) {
    if (!line) return BIN_UNKNOWN;
    if (strstr(line, "AddExpr")) return BIN_ADD;
    if (strstr(line, "SubExpr")) return BIN_SUB;
    ...
    fprintf(stderr, "[FLOW] deduce_binary_kind got line=%s -> UNKNOWN\n", line);
    return BIN_UNKNOWN;
}
```

- **Добавление узлов и рёбер CFG.** В flow.c cfg_add_node, cfg_add_edge, build_cfg_for_function управляют созданием узлов и связей. При разборе условных конструкций код вызывает cfg_add_edge с метками true/false, пример:

```
cfg_add_edge(b->cfg, cond_id, then_entry, "true");
cfg_add_edge(b->cfg, cond_id, else_entry, "false");
if (then_exit >= 0) cfg_add_edge(b->cfg, then_exit, join, NULL);
if (else_exit >= 0) cfg_add_edge(b->cfg, else_exit, join, NULL);
```

- **Связывание CFG и функций.** Финальный обработчик `Lab2/main.c` собирает `funcDef`-узлы и вызывает `build_cfg_for_function`, затем добавляет CFG в `ProgramFunction` и формирует DOT/CSV для `call-graph`:

```
for (int fi = 0; fi < func_n; fi++) {
    CFG *cfg = NULL;
    char out_fname[256];
    build_cfg_for_function(source, funcs[fi].node, &cfg, out_fname, sizeof(out_fname), prefix);
    funcs[fi].meta.cfg = cfg;
}
```

Этот цикл одновременно строит список имён (для `call-graph`) и сохраняет CFG, которые позже визуализируются.

- **Глобальный `call-graph`.** После генерации CFG `main.c` собирает все упоминания `Call(...)` из строк узлов (`scan_line_for_calls`) и формирует пары `caller→callee`, которые записываются и в `*.callgraph.dot`, и в `*.callgraph.csv`, обеспечивая граф вызовов между всеми подпрограммами.
- **Исходные деревья операций и ошибки.** `cfg_node_add_line` сохраняет оригинальные строки из CFG-узлов, что позволяет отслеживать дерево операций и показывает, если операция не распознана (`FLOW_EMIT/DEBUG`). Ошибки синтаксического анализа или чтения файлов выводятся в `stderr` (например, `Parse failed for ..., Cannot read ...`), удовлетворяя требования выводить ошибки тестовой программой.

Отчет о тестировании по критериям

Часть 3 — структуры данных

`Lab2/flow.h` содержит ключевые структуры: `ProgramFunction` (имя, сигнатура, имя файла, `CFG *cfg`), `CFG` (массив `CFGNode`), `CFGNode` (идентификатор, роль, списки `succ`, `ops`, массив `FlowOperation *flow_ops`) и `FlowOperation` (тип операции, `ExprInfo` для `lhs`, `rhs`, `call_args`, `BinaryKind/CondKind`). `ExprInfo` несёт текст, идентификатор и литерал; `BinaryKind/CondKind` помогают идентифицировать арифметические и логические выражения, а `cfg_node_add_operation` связывает операцию с узлом.

Часть 4 — интерфейс и реализация

`Lab2/main.c` реализует CLI `lab2_cfg` с опциями `--outdir`, `--emit-cfg`, `--asmout` и принимает входные файлы. Он запускает `Tree-sitter` (`tree_sitter_v2lang_test()`), находит `funcDef`, заполняет `ProgramFunction`, вызывает `build_cfg_for_function`, добавляет CFG в `ProgramImage`, а при флаге `--emit-cfg` пишет DOT-графы (`Lab2/out/*.dot`). Модуль также использует `scan_line_for_calls/CallCollectorCtx` для извлечения пар `caller→callee` и генерации `*.callgraph.dot/.csv`, а `Lab2/generate_cfgs.sh` компилирует `lab2_cfg`, очищает `Lab2/out`, парсит примеры и вызывает `dot` для PDF.

Часть 5 — примеры и результаты

`Lab2/generate_cfgs.sh` выполняет полный прогон `Lab1/examples/*.txt`. Примеры:

- `all_structures_and_expr.txt`: ветвления, циклы, массивы, выражения — DOT отображает `if_statement`, `while`, `repeat` и операторы; PDF показывает разветвления (есть в `pic/`).
- `functions.txt`, `fib_recur.txt`, `echo.txt`: функции, рекурсия, литералы — CFG отображает `funcDef/exprList`, `call-graph` фиксирует вызовы.
- `type_correct.txt` и `type_errors.txt`: корректные и некорректные декларации/присваивания; `lab2_cfg` выдает диагностические сообщения (`stderr`) для `type_errors`, остальные файлы продолжают генерироваться.
- `exmpl.txt`: подробный `main` с вызовами `send_byte`, вложенными циклами/ветвлениями и вызовами `x()`, `y()`, `z()`; файлы `Lab2/out/exmpl.txt.dot/.pdf` и `pic/exmpl_cfg.pdf/pic/exmpl_callgraph.pdf` в визуализации подтверждают структуру CFG и вызовов.
- `my_test.txt`, `fib_return.txt`, `type_errors.txt`: дополнительные кейсы проверяют литералы, массивы и сложные выражения; `.dot/.pdf` вместе с `callgraph` показывают устойчивость анализа.

Результаты

- Скомпилирован исполняемый `lab2_cfg` (сборка `Lab2/main.c`, `Lab1/src/parser.c`, `vendor/tree-sitter/lib`).
- По каждому входному файлу в `Lab2/out/*.dot` (и при наличии Graphviz, `*.pdf`) построен CFG, снабжённый текстовыми операциями и цветовой разбивкой по блокам.
- Сформированы глобальные графы: `Lab2/out/all_functions.dot/.pdf` и `*.callgraph.dot/.callgraph.csv`, отражающие структуру вызовов между подпрограммами.

Результаты тестирования

`Lab2/generate_cfgs.sh` компилирует `lab2_cfg`, парсит примеры из `Lab1/examples/*.txt`, сохраняет CFG в `Lab2/out`, собирает граф вызовов и запускает `dot` для генерации PDF (если доступен). Скрипт выводит `Wrote ...` для всех `*.dot`, `*.pdf` и `*.callgraph.*`, а `all_functions.*` собирается из подграфов `source.function.dot`.

Тестовый пример: `Lab1/examples/exmpl.txt`

Пример `exmpl.txt` содержит `method x, y, z` и `main` с вложенными циклами (`while`, `repeat`), каскадами `if/else if/else` и множественными вызовами `send_byte`. Пример проходит весь анализ, результаты сохраняются в `Lab2/out/exmpl.txt.dot` и `Lab2/out/exmpl.txt.pdf`, а глобальный `call-graph` фиксирует связи между `main` и `x/y/z`. Для визуального подтверждения можно посмотреть `pic/exmpl_cfg.pdf` и `pic/exmpl_callgraph.pdf`, где:

- базовые блоки обозначены узлами с текстовыми операциями/условиями;
- рёбра подписаны `true/false`, повторяющие циклы прослежены;
- связи вызовов показывают, как `main` обращается к вспомогательным методам, доказывая соответствие CFG AST-структуре.

Примеры исходных текстов

- `Lab1/examples/all_structures_and_expr.txt` покрывает ветвления, циклы, массивы и сложные выражения, поэтому CFG показывает разветвления и связь операций внутри блоков.
- `Lab1/examples/functions.txt`, `fib_recur.txt`, `echo.txt` дают функции, рекурсию и литералы, что подтверждает корректность `call-graph` и соответствие узлов CFG.
- `Lab1/examples/type_correct.txt / type_errors.txt` используются для проверки устойчивости анализа: корректные программы дают графы, а ошибки фиксируются в `stderr`, и `lab2_cfg` пропускает или записывает сообщения, не прерывая генерацию других файлов.

Выводы

- Построен модуль CFG, который по деревьям разбора из первой лабораторной формирует `CFGNode` с операциями (`FlowOperation`) и соединяет их в граф с условными и безусловными переходами; специфика реализации изложена в `flow.h/flow.c`.
- Тестовая программа `lab2_cfg` получает имена файлов, запускает синтаксический анализ и строит CFG, а `Lab2/generate_cfgs.sh` выполняет массовую генерацию `.dot`, `.pdf` и `call-graph` для всех входов.
- Полученные артефакты (`Lab2/out/*.dot`, `.pdf`, `.callgraph.*`, `pic/exmpl_*`) подтверждают, что CFG отражают синтаксис `exmpl.txt` и других примеров, и позволяют визуально свериться с деревьями разбора.